Isaac ROS environment setup

Isaac ROS environment setup

1. Scope
2. Obtain and check the delivered files
    Disk space requirements
3. Import the Docker image
4. Extract the course workspace
5. Start the container
    5.1 Prepare Fast DDS and the terminal environment
    5.2 Create the startup script
6. Check the ROS environment
7. Rules for preparing the models
8. Optional: delete the archives after the deployment is confirmed
9. Points to note when using it
10. Next steps

1. Scope

This tutorial is for deploying the Isaac ROS environment and the case workspace provided by the course on a Jetson Orin. Before deploying, confirm that the device matches the table below; for other Jetson models or JetPack versions, check the image compatibility first.

Item	Requirement
Mainboard	Jetson Orin, aarch64
JetPack / L4T	JetPack 7.2.1 / L4T R39.2.1
System	Ubuntu 24.04
Docker	Installed and runnable by the current user
NVIDIA Container Toolkit	Installed
Disk space	Must hold the imported image, the extracted workspace and the runtime headroom at the same time; check it with the formula in this section

The environment consists of two parts: the Docker image provides the ROS 2 Jazzy and Isaac ROS software; the workspace archive provides the course models, data and case files. The cases can only be run after both parts have been deployed.

2. Obtain and check the delivered files

Put the following two files in the `/home/jetson/` directory of the Jetson:

File	Purpose
`my-isaac-ros-slam-4.6-all.tar.gz`	The Isaac ROS Docker image; the archive is about 24.3GB and the imported Docker image is about 65.3GB
`isaac_ros_workspaces.tar.gz`	The case workspace, models and data; the package size and the extracted size vary with the delivered content, so rely on the actual files and the delivery manifest

If the files are on another computer, you can upload them from the terminal of that computer; replace `<Jetson_IP>` with the actual IP address of the Jetson:

```
scp my-isaac-ros-slam-4.6-all.tar.gz jetson@<Jetson_IP>:/home/jetson/
scp isaac_ros_workspaces.tar.gz jetson@<Jetson_IP>:/home/jetson/
```

Then confirm the files, the system and the disk space on the Jetson:

```
uname -m
cat /etc/nv_tegra_release
docker --version
docker info --format 'DockerRootDir={{.DockerRootDir}} DefaultRuntime={{.DefaultRuntime}}
Runtimes={{json .Runtimes}}'
df -h /var/lib/docker /home/jetson
ls -lh /home/jetson/my-isaac-ros-slam-4.6-all.tar.gz
/home/jetson/isaac_ros_workspaces.tar.gz
```

The device should be `aarch64` and the L4T version should be `R39.2.1`. The Docker runtime list should contain `nvidia`. If the system version, Docker or the files do not meet the requirements, contact the device/course provider first; do not replace the image or modify the global Docker configuration on your own.

Disk space requirements

First use the `ls -lh` command above to check the size of the two archives. The exact expanded size of the workspace archive should be stated by the provider in the delivery manifest. Once the archives are on the Jetson, check the remaining space with `df -h`; before starting the import, the partition holding the Docker data directory and the partition holding the workspace target should separately satisfy:

- If both are on the same partition: the remaining space has to be at least **65.3GB + the extracted workspace size + 20GB of runtime headroom**.
- If they are on different partitions: reserve at least **85GB** on the partition holding the Docker data directory, and at least **the extracted workspace size + 10GB** on the partition holding the workspace.

`df` shows the space that remains after the archives have been placed. If there is not enough space, expand it or adjust the delivery/storage location first and do not start the import or the extraction; delete the archives afterwards as described below, once the installation has been verified.

3. Import the Docker image

Run the following on the Jetson host:

```
docker load -i /home/jetson/my-isaac-ros-slam-4.6-all.tar.gz
docker image inspect my-isaac-ros-slam:4.6-all --format 'OS={{.Os}} Arch={{.Architecture}}'
docker images | grep my-isaac
```

After the import completes you should see the image `my-isaac-ros-slam:4.6-all` with the architecture `arm64`. Loading the image takes a while; do not close the terminal or delete the archive in the middle of it.

Expected output:

```
OS=linux Arch=arm64
my-isaac-ros-slam:4.6-all    xxxxxxx    about 65.3GB
```

4. Extract the course workspace

First look at the directory structure inside the archive:

```
tar -tzf /home/jetson/isaac_ros_workspaces.tar.gz | head -n 30
```

The archive should contain the complete `workspaces/isaac_ros-dev/` workspace, including at least:

- `isaac_ros_assets/`: the course models and the rosbag data
- `launch/`: the case launch files
- `rviz/`: the RViz configuration
- `scripts/` and the case helper scripts in the workspace root
- `fastdds_no_shm.xml`: the Fast DDS configuration used by the course

If the manifest is missing any of the above directories or files, obtain the complete workspace archive from the course provider first. Choose the matching command based on the result of `tar -tzf`. If the paths inside the archive start with `workspaces/isaac_ros-dev/`, extract them into `/home/jetson`:

```
mkdir -p /home/jetson/workspaces
tar -xzf /home/jetson/isaac_ros_workspaces.tar.gz -C /home/jetson
test -d /home/jetson/workspaces/isaac_ros-dev/isaac_ros_assets
test -d /home/jetson/workspaces/isaac_ros-dev/launch
test -d /home/jetson/workspaces/isaac_ros-dev/rviz
```

If the paths inside the archive start directly with `isaac_ros-dev/`, extract them into `/home/jetson/workspaces` and check:

```
mkdir -p /home/jetson/workspaces
tar -xzf /home/jetson/isaac_ros_workspaces.tar.gz -C /home/jetson/workspaces
test -d /home/jetson/workspaces/isaac_ros-dev/isaac_ros_assets
test -d /home/jetson/workspaces/isaac_ros-dev/launch
test -d /home/jetson/workspaces/isaac_ros-dev/rviz
```

Before extracting, confirm that the target directory does not exist or does not contain files that have to be kept; if the check fails, first verify the directory structure of the archive and do not start the container.

5. Start the container

5.1 Prepare Fast DDS and the terminal environment

The workspace is located in a directory mounted from the host, so putting the configuration file in the workspace keeps it across container restarts. First check whether the workspace already has `fastdds_no_shm.xml`; if not, create it on the Jetson host:

```
cat > /home/jetson/workspaces/isaac_ros-dev/fastdds_no_shm.xml <<'EOF'
<?xml version="1.0" encoding="UTF-8" ?>
<profiles xmlns="http://www.eprosima.com/XMLSchemas/fastRTPS_Profiles">
  <transport_descriptors>
    <transport_descriptor>
      <transport_id>udp_only</transport_id>
      <type>UDPv4</type>
    </transport_descriptor>
  </transport_descriptors>
  <participant profile_name="udp_only_participant" is_default_profile="true">
    <rtps>
      <userTransports>
        <transport_id>udp_only</transport_id>
      </userTransports>
      <useBuiltinTransports>false</useBuiltinTransports>
    </rtps>
  </participant>
</profiles>
EOF
```

The Fast DDS configuration uses the UDP transport, which is the configuration for the container environment of this course. The startup script generates `/tmp/.bashrc` every time it starts the container, and it only contains the ROS and workspace environment settings; there is no need to create another environment file on the host, and the user configuration inside the image is not modified.

5.2 Create the startup script

On the first use, create the startup script on the Jetson host:

```
cat > /home/jetson/run_isaac.sh <<'EOF'
#!/bin/bash
set -e

ISAAC_ROS_WS="$HOME/workspaces/isaac_ros-dev"

if [[ ! -d "$ISAAC_ROS_WS" ]]; then
  echo "Workspace does not exist: $ISAAC_ROS_WS" >&2
  exit 1
fi
if [[ ! -f "$ISAAC_ROS_WS/fastdds_no_shm.xml" ]]; then
  echo "Fast DDS configuration file does not exist: $ISAAC_ROS_WS/fastdds_no_shm.xml" >&2
  exit 1
fi
```

```
docker run --rm -it \
  --runtime=nvidia --gpus all \
  --network host --ipc host --privileged \
  -u 1000:1000 -e HOME=/tmp \
  -e DISPLAY="$DISPLAY" -e QT_X11_NO_MITSHM=1 \
  -v /tmp/.X11-unix:/tmp/.X11-unix:rw \
  -v "$ISAAC_ROS_WS:/workspaces/isaac_ros-dev" \
  --entrypoint /bin/bash \
  my-isaac-ros-slam:4.6-all \
  -c '
cat > /tmp/.bashrc <<'"'"'BASHRC_EOF'"'"'
source /opt/ros/jazzy/setup.bash
export ISAAC_ROS_WS=/workspaces/isaac_ros-dev
export FASTRTPS_DEFAULT_PROFILES_FILE=/workspaces/isaac_ros-dev/fastdds_no_shm.xml
BASHRC_EOF
exec /bin/bash --rcfile /tmp/.bashrc -i
'
EOF
```

```
chmod +x /home/jetson/run_isaac.sh
```

The script is written for the `jetson` user with the UID/GID `1000:1000`; other users should run `id` first and then change the `-u` argument to match the actual result. When the container starts it writes the ROS and workspace environment into `/tmp/.bashrc` and starts an interactive Bash with that file through `--rcfile`. `--privileged` is part of the configuration of the current course device and is not recommended for containers used for other purposes.

The graphical cases have to be started from a terminal on the Jetson desktop. First allow the local Docker containers to access the current desktop, and then run the startup script:

```
xhost +local:docker
cd /home/jetson
./run_isaac.sh
```

After all the graphical programs have been closed, you can revoke that permission:

```
xhost -local:docker
```

Do not use `xhost +` to switch off all the graphical access control, and do not modify the global Docker runtime on your own. If an error is reported at startup, record the complete error message and check the image, the workspace and the desktop session configuration.

After the container has started, open another Jetson terminal to see the container name:

```
docker ps --format 'table {{.Names}}\t{{.Image}}\t{{.Status}}'
```

When you need an additional terminal in the container, run the following on the Jetson host:

```
docker exec -it <container name> bash
```

Replace `<container name>` with the name shown by `docker ps`.

6. Check the ROS environment

When you enter the container with the startup script of this tutorial, the ROS and Fast DDS environment is loaded automatically from `/tmp/.bashrc`. Run the following commands inside the container to check it:

```
echo "$ROS_DISTRO"
echo "$ISAAC_ROS_WS"
echo "$FASTRTPS_DEFAULT_PROFILES_FILE"
test -f "$FASTRTPS_DEFAULT_PROFILES_FILE"
test -d "$ISAAC_ROS_WS/isaac_ros_assets"
test -d "$ISAAC_ROS_WS/launch"
```

The expected ROS distribution is `jazzy`, the workspace is `/workspaces/isaac_ros-dev` and the Fast DDS configuration file exists. Then check the core package:

```
ros2 pkg prefix isaac_ros_visual_slam
```

The expected package path is under `/opt/ros/jazzy`. If you start the container in another way, you have to run `source /opt/ros/jazzy/setup.bash` manually in each new terminal and set `ISAAC_ROS_WS` and `FASTRTPS_DEFAULT_PROFILES_FILE`.

7. Rules for preparing the models

Case 5 uses the TensorRT plan in the `optimized_deployable_shuffleseg_unet_amr_v1.0` directory, which the launch file loads directly; case 7 uses the Triton model `peoplenet`. Cases 1-4, 6, 8 and 9 do not have to be switched in this way. Before switching, stop the current case, check the two directories inside the container and then run the following as needed:

```
: cd /workspaces/isaac_ros-dev/isaac_ros_assets
ls models/triton/ models_pending/

# Move all the current models under models/triton/ back to the pending directory
mv models/triton/* models_pending/

# Replace <model name> with the directory name corresponding to the model above
mv models_pending/<model name> models/triton/
ls models/triton/
```

`mv models/triton/*` moves away all the models in the active directory, so run it only when you are sure that no other model has to be kept; if the target model is already in the active directory, no switch is needed. Before moving, confirm that there is no directory with the same name in pending; if the command reports an error, check the directories first instead of hiding the error with `2>/dev/null`. The CenterPose FP32 plan of case 8 is specified by the startup script and is not switched here.

8. Optional: delete the archives after the deployment is confirmed

After you have confirmed that the image has been imported successfully, that the workspace has been extracted and that the container and the cases start normally, you can delete the two archives if you do not need to keep the installation packages. This operation does not delete the imported Docker image or the extracted workspace files; keep the archives until the deployment has been confirmed.

```
rm /home/jetson/my-isaac-ros-slam-4.6-all.tar.gz
rm /home/jetson/isaac_ros_workspaces.tar.gz
```

9. Points to note when using it

- Run only one heavy case at a time; start the next one after the current one has finished.
- Start and stop the nodes of each case according to its own instructions, and exit normally with `Ctrl+C` in the corresponding terminal; do not use `clean_ros` or a broad `pkill` to end the processes in bulk.
- The container itself is removed automatically when it exits; the Docker image and the files in the host directory `/home/jetson/workspaces/isaac_ros-dev/` are kept.
- If you get `groups: cannot find name for group ID`, but the container, the ROS nodes and the case check are all normal, this message usually does not affect the run.
- If RViz or rqt cannot display anything, confirm that it was started from a terminal on the Jetson desktop and check that the desktop graphical permission has been set.
- If Docker, the image import or the workspace check fails, record the complete error message and contact the course provider; do not delete the delivered archives or modify the system-level Docker configuration.

10. Next steps

Once the environment check has passed, run the course content by following the instructions of cases 1-9 in the directory of this chapter.